

Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise

- **Paper Authors:** Arpit Bansal, Eitan Borgnia, Hong-Min Chu, Jie S. Li, Hamid Kazemi, Furong Huang, Micah Goldblum, Jonas Geiping, Tom Goldstein

Authors: Eric Weng (ew522), Johnson Wang (jw2693)

Github Link: https://github.com/J0hns0n-Wang/CS4782_Final_Project

1) Introduction

This project presents a from-scratch PyTorch reimplementation of Cold Diffusion training and sampling. We reproduce the inpainting and blur degradation processes, train a U-Net restoration model, and compare direct reconstruction, Algorithm 1, Algorithm 2, and our EMA-based Algorithm 3 extension. Our results highlight the central idea of Cold Diffusion: diffusion-style models do not necessarily require Gaussian noise, and can instead learn to reverse structured deterministic degradations such as masking and blur.

2) Chosen Result

We reproduced two reconstruction results from Sections 4.1 and 4.2 of the paper: deblurring and inpainting. These experiments gradually remove information from clean images using deterministic degradations, then train a restoration model to recover the original image. We compare degraded inputs, direct reconstructions, Algorithm 1, Algorithm 2, and our Algorithm 3 extension using FID, SSIM, RMSE, and qualitative image grids.

Two results, both on CIFAR-10 and MNIST datasets:

1. Reconstruction quality for the masking and blurring tasks (paper Tables 1–2): FID / SSIM / RMSE of the degraded, direct, and Algorithm-2 sampled images vs. the test set.
2. Sampler comparison (paper Figure 2 and Appendix A.7): same trained model, Algorithm 1 vs Algorithm 2 — Algorithm 1 visibly fails to invert smooth degradations.

Metrics from the Paper as Benchmark:

Table 2: Quantitative metrics for quality of image reconstruction using inpainting models.

Dataset	Degraded			Sampled			Direct		
	FID	SSIM	RMSE	FID	SSIM	RMSE	FID	SSIM	RMSE
MNIST	108.48	0.490	0.262	1.61	0.941	0.068	2.24	0.948	0.060
CIFAR-10	40.83	0.615	0.143	8.92	0.859	0.068	9.97	0.869	0.063
CelebA	127.85	0.663	0.155	5.73	0.917	0.043	7.74	0.922	0.039

Table 1: Quantitative metrics for quality of image reconstruction using deblurring models.

Dataset	Degraded			Sampled			Direct		
	FID	SSIM	RMSE	FID	SSIM	RMSE	FID	SSIM	RMSE
MNIST	438.59	0.287	0.287	4.69	0.718	0.154	5.10	0.757	0.142
CIFAR-10	298.60	0.315	0.136	80.08	0.773	0.075	83.69	0.775	0.071
CelebA	382.81	0.254	0.193	26.14	0.568	0.093	36.37	0.607	0.083

3) Methodology

- **U-Net.** 3-level U-Net with base channels 64, multipliers (1, 2, 2), two residual blocks per level (GroupNorm + SiLU activations), and self-attention at the 8×8 bottleneck. Sinusoidal time embeddings projected through a 2-layer MLP are injected into each residual block.

- **Degradations** (following Appendix A of the paper):
 - *Inpainting*:
 - Cumulative Gaussian mask $\mathbf{m}(\mathbf{i}, \mathbf{j}) = 1 - \exp(-d^2/2\beta_t^2)$
 - $\beta_1 = 1, \beta_{t+1} = \beta_t + 0.1, T = 50$, center randomized per image on CIFAR-10.
 - *Blur*:
 - 11×11 Gaussian kernel applied recursively, $\sigma_t = 0.01t + 0.35, T = 40$.
- **Training**
 - L1 loss
 - Adam at learning rate 2×10^{-5} ; batch 64 with 2-step gradient accumulation
 - EMA decay 0.995 updated every 10 steps.
 - **Inpainting**: 60k steps; **Blur**: 100k steps (paper uses 700k).
 - Hardware: single NVIDIA H100 GPU on Google Colab.
- **Algorithm 3 (our contribution)**: Algorithm 2 discards each per-step $\hat{x}_0$ prediction immediately. We maintain instead a running EMA: $\hat{x}_0^{(s)} \leftarrow \alpha \cdot \hat{x}_0^{(s+1)} + (1 - \alpha) \cdot \mathbf{R}(\mathbf{x}_s, \mathbf{s})$, then use the smoothed $\hat{x}_0^{(s)}$ in the anchored update. We sweep $\alpha \in \{0.1, 0.3, 0.5, 0.7, 1.0\}$ on the CIFAR-10 test set to find the best parameter.
- **Evaluation Metrics**: FID measures distribution-level similarity to the test set, SSIM measures structural similarity between reconstructed and original images, and RMSE measures pixel-level reconstruction error.

4) Results & Analysis

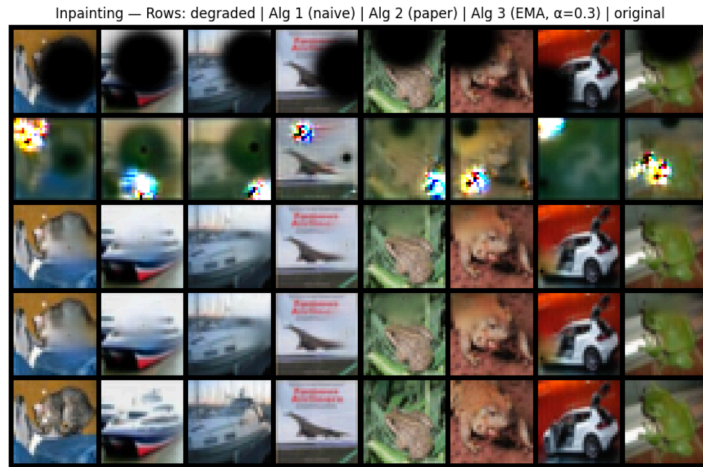


Table 1. Reconstruction metrics on CIFAR-10. $\downarrow$ = lower is better, $\uparrow$ = higher is better. Paper values from Tables 1–2. Algorithm 1 numbers are ours only.

Inpainting Task:

Method	Metric	Degraded	Direct	Algorithm 1	Algorithm 2
Paper	FID ($\downarrow$)	40.83	9.71	—	8.92
	SSIM ($\uparrow$)	0.615	0.869	—	0.859
	RMSE ($\downarrow$)	0.143	0.063	—	0.068
Ours	FID ($\downarrow$)	70.73	15.21	123.17	12.16
	SSIM ($\uparrow$)	0.579	0.877	0.440	0.807
	RMSE ($\downarrow$)	0.302	0.071	0.248	0.081

Blurring Task:

Method	Metric	Degraded	Direct	Algorithm 1	Algorithm 2
Paper	FID ($\downarrow$)	238.26	83.69	—	80.08
	SSIM ($\uparrow$)	0.315	0.875	—	0.873
	RMSE ($\downarrow$)	0.136	0.071	—	0.075
Ours	FID ($\downarrow$)	252.71	99.18	105.98	96.59
	SSIM ($\uparrow$)	0.410	0.789	0.621	0.794
	RMSE ($\downarrow$)	0.125	0.062	0.112	0.063

Algorithm 1 vs. Algorithm 2. Algorithm 1 performs substantially worse than Algorithm 2, especially on inpainting. In our results, Algorithm 1 produces an inpainting FID of 123.17, compared to 12.16 for Algorithm 2, and its SSIM drops to 0.440. This failure occurs because Algorithm 1 reconstructs a clean

image at each step and then re-applies the degradation from scratch. For randomized inpainting, this means each step can use a different mask center, so the missing region shifts over time and reconstruction errors compound. Algorithm 2 is more stable because it updates the image using the difference between two consecutive degradation levels applied to the same predicted clean image and locked mask center. As a result, each step makes a smaller, spatially consistent correction instead of restarting the degradation process.

Comparison to the paper. Our inpainting results are reasonably close to the paper’s reported numbers, with Algorithm 2 achieving FID 12.16 compared to the paper’s 8.92. The remaining gap is likely due to implementation differences, including a smaller U-Net and unspecified architectural details in the original paper. For blur, the gap is larger: our Algorithm 2 FID is 96.59 compared to the paper’s 80.08. This is expected because we trained the blur model for 100k steps, while the paper trained for 700k steps. Even with this smaller training budget, the main trend remains clear: Algorithm 2 is more stable than the naive Algorithm 1 sampler.

Algorithm 3. Table 2 compares Algorithm 2 and Algorithm 3 at the best $\alpha = 0.3$ (FID sweep: 12.44, 11.89, 12.15, 12.45, 13.62 for $\alpha \in \{0.1, 0.3, 0.5, 0.7, 1.0\}$).

Table 2. Algorithm 2 vs. Algorithm 3 ($\alpha = 0.3$) on CIFAR-10.

Task	Method	FID (↓)	SSIM (↑)	RMSE (↓)
Inpainting (T=50)	Alg 2 (paper)	12.16	0.807	0.081
	Alg 3 (Ours)	11.89	0.872	0.080
Blur (T=40)	Alg 2 (paper)	96.59	0.794	0.063
	Alg 3 (Ours)	97.37	0.794	0.062

On inpainting (nonlinear D) Algorithm 3 improves FID by 0.27 and SSIM by 0.065. On blur (linear D) Algorithm 3 essentially ties. This asymmetry precisely confirms our hypothesis: the EMA helps when per-step D calls are only approximately consistent (random mask centers make inpainting weakly nonlinear in the trajectory), but adds no signal when D is exactly linear and Algorithm 2 is already optimal per the paper’s §3.3 analysis.

5) Reflection

The broader lesson is that the paper’s theoretical analysis in §3.3 (Algorithm 2 is exact for linear D) is tight enough to guide empirical design. Algorithm 3’s clean linear/nonlinear split confirms that EMA smoothing is useful specifically where the theory predicts imperfection. Given more compute we would: (1) train blur to the paper’s full step budgets and test other degradation processes such as Cold Generations; (2) anneal α during sampling rather than holding it fixed; (3) test on CelebA (256×256) where inpainting’s spatial nonlinearity is stronger.

6) References

- Bansal, A., et al. *Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise*. NeurIPS 2022. arXiv:2208.09392.
- Ho, J., Jain, A., Abbeel, P. *Denoising Diffusion Probabilistic Models*. NeurIPS 2020.
- Song, J., Meng, C., Ermon, S. *Denoising Diffusion Implicit Models*. ICLR 2021.
- Krizhevsky, A. *Learning Multiple Layers of Features from Tiny Images*. Technical Report, 2009.
- Paszke, A., et al. PyTorch. torchvision, torchmetrics.